BIOINFORMATICS MACHINE LEARNING BOOTCAMP

Machine Learning et Deep Learning pour les biologistes

De la donnée brute au modèle prédictif reproductible

Auteur : **DEDJI ABDOUL BATINOU**

Marque éditoriale : Omicaa Lab

Ebook accompagné de sept notebooks Jupyter pour Google Colab

Dépôt : github.com/DEDJIABDOUL/omicaa-lab-machine-learning-bioinformatics (à publier)

SOMMAIRE

Table des matières

Cadre, méthode et architecture du parcours 5

1.1 Un pipeline unique, du fichier au modèle 5

1.2 Trois couches de compétences 5

1.3 Les deux jeux de données et leur justification 5

1.4 Reproductibilité 6

Python scientifique minimal 7

2.1 Variables, types et chaînes 7

2.2 Fonctions, arguments et valeur de retour 7

2.3 Exceptions, fichiers et lecture d'une erreur 7

Représentation numérique des données 8

3.1 Tableaux NumPy 8

3.2 DataFrames pandas 8

3.3 Observation, caractéristique, étiquette, mesure 8

3.4 Décider du codage de la cible 9

Analyse exploratoire et contrôle qualité 9

4.1 Une séquence d'exploration en neuf étapes 9

4.2 Distribution de la cible et des variables 9

4.3 Corrélations et redondance 10

4.4 Anomalie technique, valeur atypique, observation plausible 11

Formalisation, prétraitement et partitionnement 11

5.1 Formaliser le problème d'apprentissage 11

5.2 Prétraitement : des transformations qui apprennent 12

5.3 Partitionner sans fuite 12

5.4 Observations groupées 13

Modélisation et comparaison algorithmique 13

6.1 Le modèle de référence 13

6.2 Quatre algorithmes et leurs propriétés 13

6.3 Assembler des pipelines 14

6.4 Résultats sur un jeu de validation 14

Évaluation expérimentale 15

7.1 Matrice de confusion : un scénario biologique 15

7.2 Définitions des métriques 15

7.3 Choisir et justifier ses métriques . . . 16

Validation croisée, optimisation et évaluation finale . . . 17

8.1 Validation croisée stratifiée . . . 17

8.2 Rechercher les hyperparamètres . . . 18

8.3 Validation croisée imbriquée . . . 18

8.4 Observations groupées : démonstration . . . 18

8.5 Règle de sélection, seuil et évaluation unique . . . 19

8.6 Sauvegarder, recharger, prédire . . . 20

Introduction opérationnelle au deep learning . . . 21

9.1 Quand le deep learning se justifie . . . 21

9.2 Neurone, perte et gradient . . . 21

9.3 Construire, entraîner, surveiller . . . 22

9.4 Diagnostiquer le surapprentissage . . . 22

9.5 Comparaison dans des conditions compatibles . . . 23

Projet intégrateur : expression génique et leucémies . . . 23

10.1 Contexte et contrôles d'entrée . . . 23

10.2 La fuite par sélection de gènes, démontrée . . . 24

10.3 Comparaison par validation croisée imbriquée . . . 25

10.4 Évaluation finale et lecture prudente . . . 25

10.5 Stabilité de la sélection et limites . . . 26

Reproductibilité, interprétation scientifique et suite . . . 27

11.1 Procédure de validation du code . . . 27

11.2 Trois niveaux d'interprétation . . . 27

11.3 Limites de ce dispositif . . . 27

11.4 Pour aller plus loin . . . 28

MODE D'EMPLOI

Comment utiliser ce dispositif

Ce document est le texte de référence d'un parcours d'ingénierie appliquée. Il accompagne sept notebooks Jupyter, exécutables dans Google Colab, et un dépôt GitHub. L'apprenant y construit, étape par étape, un système prédictif à partir de données biologiques : il comprend chaque transformation, chaque choix algorithmique et chaque résultat. Il ne s'agit ni d'une synthèse théorique du machine learning ni d'une collection de tutoriels isolés.

Public visé. Un biologiste ou un étudiant en génomique, bioinformatique ou sciences biomédicales qui ne connaît ni Python, ni NumPy, ni pandas, ni scikit-learn. Des connaissances élémentaires de biologie et de statistiques descriptives suffisent. Aucune algèbre linéaire avancée n'est supposée.

Principe pédagogique. Le parcours repose sur un projet fil rouge, le diagnostic de tumeurs à partir de mesures de noyaux cellulaires, puis sur un second projet transcriptomique qui vérifie la capacité de transfert des compétences. Les chapitres du livre et les notebooks se répondent : le livre explique le pourquoi, les notebooks permettent de le faire.

Notebook	Contenu	Chapitres
01 Python fondamentaux	variables, fonctions, FASTA, exceptions	2
02 NumPy, pandas, exploration	tableaux, DataFrames, contrôle qualité initial	3, 4
03 Visualisation et prétraitement	figures, valeurs atypiques, imputation, partitionnement, fuite	4, 5
04 Classification et pipelines	modèle de référence, quatre algorithmes, métriques	6, 7
05 Validation et optimisation	validation croisée, recherche d'hyperparamètres, test final	8
06 Deep learning avec TensorFlow	neurone, gradient, réseau Keras, surapprentissage	9
07 Projet intégrateur	expression génique de leucémies, sélection de gènes sans fuite	10

Tableau 1. Correspondance entre notebooks et chapitres.

À RETENIR

Exécution. Ouvrir chaque notebook dans Colab, puis *Exécution*, *Tout exécuter*. Chaque notebook commence par un contrôle de l'environnement et se termine par des exercices suivis de leurs solutions.
Intégrité des résultats. Tous les chiffres de ce livre proviennent de l'exécution réelle des notebooks, avec la graine aléatoire 42. Les valeurs peuvent varier légèrement selon la version des bibliothèques et le processeur.

POINT D'ATTENTION

Ce parcours enseigne une méthode. Les modèles construits ici ne sont pas des outils de diagnostic et leurs performances ne constituent pas une validation clinique.

CHAPITRE 1

Cadre, méthode et architecture du parcours

Un modèle prédictif n'est crédible que si son protocole l'est. Ce chapitre pose l'architecture commune à tout le parcours.

1.1 Un pipeline unique, du fichier au modèle

Toute la formation suit la même chaîne de treize étapes. Chaque chapitre introduit une partie du pipeline et réutilise les acquis des précédents. L'étape d'évaluation finale est encadrée en rouge parce qu'elle obéit à une règle particulière : le jeu de test n'y est consulté qu'une seule fois.

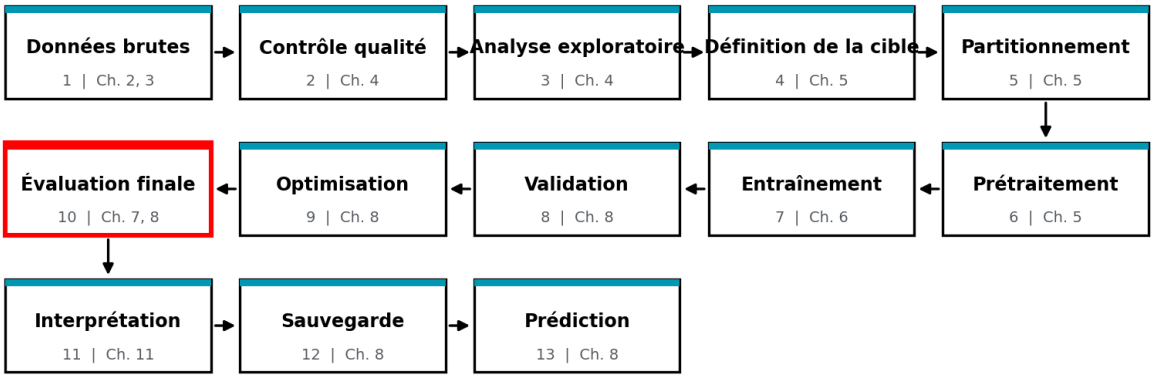


Figure 1. Le pipeline conceptuel du parcours. Les références renvoient aux chapitres qui traitent chaque étape.

1.2 Trois couches de compétences

- **Couche A, fondamentaux.** Syntaxe Python minimale, tableaux NumPy, DataFrames pandas, visualisation, compréhension de la structure des données (chapitres 2 à 4).
- **Couche B, ingénierie du machine learning.** Prétraitement, séparation des données, pipelines, algorithmes, métriques, validation croisée et optimisation (chapitres 5 à 8).
- **Couche C, extension vers le deep learning.** Tenseurs, couches, fonctions d'activation, fonction de perte, rétropropagation, régularisation (chapitre 9).

Le deep learning n'est pas présenté comme une amélioration systématique du machine learning classique. Le chapitre 9 explique dans quelles situations il se justifie et montre, sur un tableau de quelques centaines de lignes, qu'il n'apporte pas de gain démontrable.

1.3 Les deux jeux de données et leur justification

Les serveurs UCI et NCBI GEO n'étaient pas joignables depuis l'environnement de validation (réponse HTTP 403). Les jeux de données ont donc été choisis parmi des sources que le code pouvait réellement récupérer et exécuter, et chaque limite de provenance est signalée.

	Projet fil rouge	Projet de transfert
Jeu de données	Breast Cancer Wisconsin (Diagnostic)	Leucémies de Golub et al. (1999)
Origine	UCI Machine Learning Repository, identifiant 17, DOI 10.24432/C5DW2B. Copie intégrée à scikit-learn	Publication Science 286:531-537. Copie publique sur GitHub (dépôt ramhiser/datamicroarray)
Licence	CC BY 4.0 (vérifiée sur la page UCI)	Non vérifiée pour la copie utilisée : à contrôler avant redistribution
Taille	569 observations, 30 variables	72 échantillons, 7129 sondes
Cible	tumeur maligne (212) ou bénigne (357)	AML (25) ou ALL (47)
Compétences visées	pipeline complet, métriques, validation croisée, réseau dense	p très supérieur à n, sélection de variables sans fuite, test minuscule

Tableau 2. Les deux jeux de données et leurs rôles pédagogiques.

Le premier jeu est tabulaire, de taille modérée, sans valeur manquante et exécutable sur CPU : il permet d'apprendre la méthode sans la noyer sous le nettoyage. Le second concentre les difficultés de la transcriptomique, un nombre de variables environ 99 fois supérieur au nombre d'échantillons, et oblige à raisonner sur la fuite d'information. Ce choix est justifié par les compétences à acquérir et non par la popularité des données.

1.4 Reproductibilité

Chaque notebook fixe la graine 42 pour Python, NumPy et, le cas échéant, TensorFlow, affiche les versions des bibliothèques et vérifie ses entrées par des assertions. Les résultats de ce livre ont été obtenus avec Python 3.12, NumPy 2.4, pandas 3.0, scikit-learn 1.8, Matplotlib 3.10 et TensorFlow 2.21. Colab peut proposer d'autres versions : les conclusions qualitatives sont stables, les décimales peuvent légèrement différer.

> **À RETENIR**
> **Quatre règles d'or du parcours.** 1. Un modèle de référence avant tout modèle complexe. 2. Tout ce qui apprend quelque chose des données, y compris la sélection de variables, appartient au pipeline. 3. Le jeu de test n'est utilisé qu'une fois, après figement du modèle. 4. Une performance est toujours rapportée avec son incertitude et ses limites.

CHAPITRE 2

Python scientifique minimal

Le but n'est pas de former un développeur, mais de permettre au biologiste de lire, modifier et déboguer un script d'analyse.

2.1 Variables, types et chaînes

Une **variable** est un nom qui désigne une valeur. Le **type** de la valeur détermine les opérations possibles. L'objet biologique le plus naturel pour s'exercer est la séquence nucléotidique, qui est une chaîne de caractères indexée à partir de 0.

Type	Exemple	Usage biologique
int	3200	longueur d'une séquence
float	0.47	contenu en GC
str	"ATGC"	séquence, identifiant de gène
bool	True	gène exprimé ou non
NoneType	None	valeur absente

Tableau 3. Les types scalaires de Python et leur usage.

Le slicing `s[a:b]` extrait les positions de a inclus à b exclu. La liste est une collection ordonnée et modifiable, le dictionnaire associe une clé à une valeur, ce qui en fait la structure naturelle pour relier un identifiant de gène à une mesure.

2.2 Fonctions, arguments et valeur de retour

Une **fonction** encapsule une opération réutilisable. Ses **arguments** sont ses entrées, sa **valeur de retour** sa sortie. Une fonction robuste valide son entrée et lève une erreur explicite plutôt que de produire un résultat faux en silence. Le code 1 en donne un exemple sur le contenu en GC.

```
def contenu_gc(sequence: str) -> float:
    """Retourne la proportion de bases G ou C dans une séquence d'ADN.

    Lève ValueError si la séquence est vide ou contient une base inattendue.
    """
    sequence = sequence.upper()
    if len(sequence) == 0:
        raise ValueError("séquence vide")
    invalides = set(sequence) - set("ACGTN")
    if invalides:
        raise ValueError(f"bases inattendues : {sorted(invalides)}")
    return (sequence.count("G") + sequence.count("C")) / len(sequence)
```

Code 1. Fonction de contenu en GC avec validation de l'entrée (notebook 01).

La fonction est protégée par des tests automatiques (`assert contenu_gc('GGCC') == 1.0`). Cette habitude, qui consiste à écrire ce que l'on attend avant de faire confiance au résultat, structure tout le parcours.

2.3 Exceptions, fichiers et lecture d'une erreur

Le bloc `try/except` traite une situation anormale sans interrompre tout le script. Le notebook 01 lit un fichier FASTA en vérifiant sa structure : identifiants uniques, séquence précédée d'un en-tête, fichier existant. Une **trace d'erreur** se lit de bas en haut : la dernière ligne donne le type et le message de l'erreur, les lignes précédentes l'enchaînement des appels.

Erreur courante	Cause habituelle
NameError	nom non défini, souvent une cellule non exécutée
TypeError	opération sur des types incompatibles
KeyError	clé absente d'un dictionnaire ou d'un DataFrame
IndexError	position hors limites
FileNotFoundError	mauvais chemin de fichier
ZeroDivisionError	division par zéro, par exemple moyenne d'une liste vide

Tableau 4. Erreurs fréquentes d'un débutant et leur cause.

CHAPITRE 3

Représentation numérique des données

Une mesure biologique devient exploitable par un algorithme lorsqu'elle est représentée par un tableau de nombres.

3.1 Tableaux NumPy

Un `ndarray` est un tableau de valeurs de même type, stockées de façon contiguë. Il possède une **forme** (`shape`), un nombre de dimensions et un type. Une matrice d'expression de 4 échantillons et 3 gènes a la forme (4, 3) : les lignes sont les échantillons, les colonnes les gènes. Cette convention (n, p) est celle de tout le parcours.

Les opérations s'appliquent à tous les éléments sans boucle explicite. L'argument `axis` choisit le sens de l'agrégation : `axis=0` résume les lignes et donne un résultat par colonne. Lorsque deux tableaux n'ont pas la même forme, NumPy étend le plus petit si les dimensions sont compatibles : c'est le **broadcasting**. Soustraire à une matrice (4, 3) la moyenne de chaque colonne, de forme (3,), centre chaque gène.

```
moyennes = expr.mean(axis=0)
centre = expr - moyennes
print("forme moyennes :", moyennes.shape, "| forme résultat :", centre.shape)
print("moyennes après centrage :", centre.mean(axis=0).round(12))
assert np.allclose(centre.mean(axis=0), 0)
```

Code 2. Centrage par broadcasting, opération que reproduira StandardScaler (notebook 02).

3.2 DataFrames pandas

Un `DataFrame` associe à un tableau des noms de colonnes et un index de lignes, et autorise des types différents par colonne. Il porte aussi les outils du quotidien : filtrage par condition, regroupement (`groupby`), jointure (`merge`) sur une clé commune et gestion des valeurs manquantes.

3.3 Observation, caractéristique, étiquette, mesure

Terme	Définition	Dans le projet fil rouge
Observation	une ligne, une unité statistique	un échantillon cellulaire analysé
Caractéristique (feature)	une colonne d'entrée du modèle	mean radius, worst texture, ...
Étiquette (cible)	la valeur à prédire	maligne (1) ou bénigne (0)
Mesure biologique	la grandeur physique derrière la colonne	dimension d'un noyau, issue d'une image de ponction à l'aiguille fine

Tableau 5. Vocabulaire fondamental et correspondance avec le jeu de données.

Une caractéristique est une **représentation** numérique d'une mesure, non la mesure elle-même. Les 30 colonnes du jeu Wisconsin décrivent dix propriétés du noyau (rayon, texture, périmètre, aire, régularité, compacité, concavité, points concaves, symétrie, dimension fractale), chacune résumée par une moyenne, une erreur standard et une valeur extrême. Elles sont calculées sur les mêmes noyaux et donc fortement corrélées.

3.4 Décider du codage de la cible

Dans scikit-learn, la classe 0 est *malignant* et la classe 1 *benign*. Or précision, rappel et F1 considèrent par défaut la classe 1 comme positive. En clinique, on cherche à détecter la tumeur maligne : elle doit être la classe positive. Le notebook recode donc la cible, et ce choix est conservé partout.

```
y = (bunch.target == 0).astype(int).rename("maligne")
print(y.value_counts().sort_index().rename({0: "benigne (0)", 1: "maligne (1)"}))
assert set(y.unique()) == {0, 1}
assert len(X) == len(y) == 569
```

Code 3. Recodage de la cible : 1 pour maligne, avec contrôles de cohérence (notebook 02).

POINT D'ATTENTION
Un codage de cible implicite est une source classique d'erreurs d'interprétation. Écrivez toujours, dans le code et dans le texte, quelle classe est positive.

CHAPITRE 4

Analyse exploratoire et contrôle qualité

Visualiser avant de modéliser est une exigence méthodologique : un graphique révèle en quelques secondes ce que des statistiques résumées masquent.

4.1 Une séquence d'exploration en neuf étapes

Étape	Contrôle	Résultat sur le jeu Wisconsin
1	Dimensions et types	569 lignes, 30 colonnes, toutes de type flottant
2	Identifiants et doublons	aucune ligne dupliquée ; copie sans identifiant patient (limite)
3	Valeurs manquantes	aucune ; des zéros valides dans six colonnes de concavité
4	Statistiques descriptives	échelles très hétérogènes (aire moyenne 654,9, régularité moyenne 0,096)
5	Distribution des variables	décalage net entre classes pour rayon, points concaves, aire
6	Relations entre variables	21 paires de colonnes avec une corrélation absolue supérieure à 0,9
7	Valeurs atypiques	171 observations hors bornes de l'écart interquartile
8	Distribution de la cible	357 bénignes, 212 malignes (62,7 % et 37,3 %)
9	Anomalies compromettantes	aucune incohérence géométrique détectée

Tableau 6. Séquence de contrôle qualité et résultats observés (notebooks 02 et 03).

4.2 Distribution de la cible et des variables

Une règle qui prédirait toujours *bénigne* aurait une accuracy d'environ 0,627 : c'est le plancher à dépasser. Les classes sont déséquilibrées de façon modérée, ce qui justifie des partitions stratifiées.

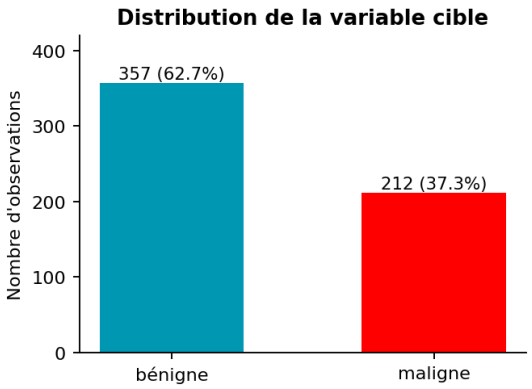


Figure 2. Distribution de la variable cible. La classe maligne, en rouge, est la classe positive.

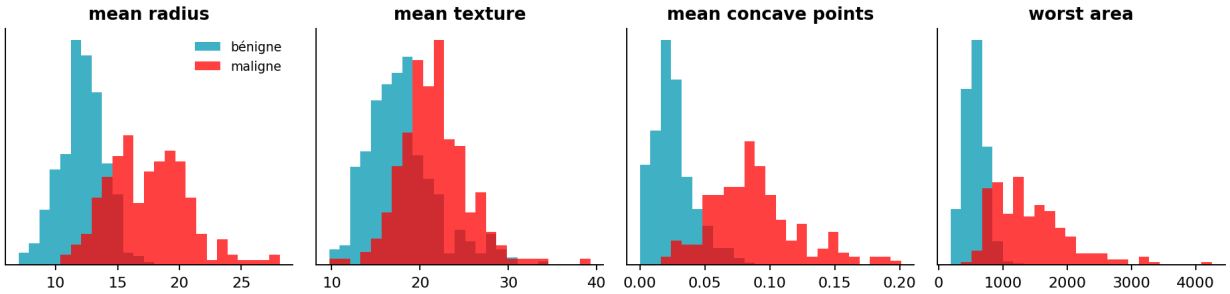


Figure 3. Distribution de quatre caractéristiques selon la classe (histogrammes normalisés). La texture sépare moins bien les classes que le rayon, les points concaves ou l'aire.

Une caractéristique dont les distributions se recouvrent reste utile en combinaison avec d'autres, mais elle sépare mal les classes à elle seule.

4.3 Corrélations et redondance

Le coefficient de Pearson mesure la dépendance linéaire entre deux variables, de -1 à 1. Rayon, périmètre et aire décrivent la même grandeur géométrique : leur corrélation dépasse 0,98. Cette redondance déstabilise les coefficients d'une régression logistique et répartit l'importance entre variables dans une forêt aléatoire, sans nécessairement dégrader la prédiction.

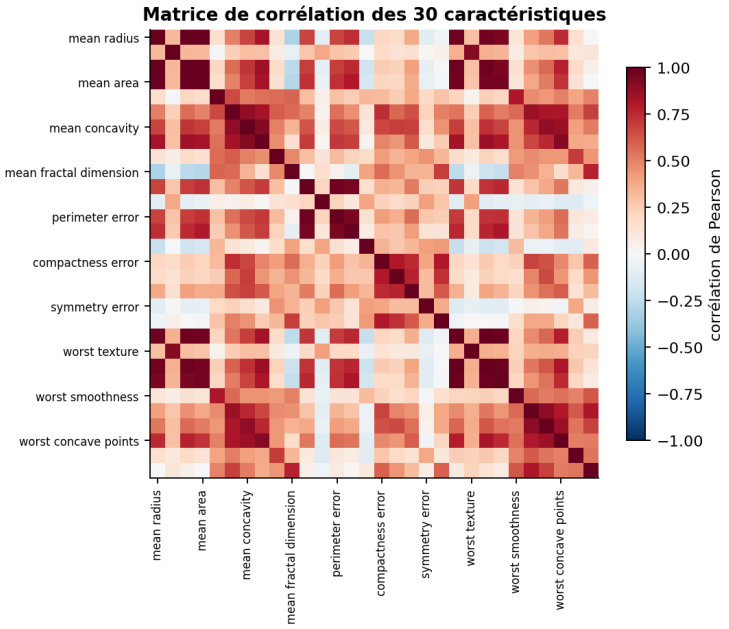


Figure 4. Matrice de corrélation des 30 caractéristiques. Les blocs rouges révèlent des groupes de mesures redondantes.

4.4 Anomalie technique, valeur atypique, observation plausible

Situation	Définition	Décision
Anomalie technique	valeur impossible ou incohérente : unité erronée, erreur de saisie, artefact	corriger, exclure ou signaler, avec traçabilité
Valeur atypique	valeur éloignée de la masse, sans impossibilité démontrée	examiner, conserver par défaut, mesurer l'effet sur le modèle
Observation plausible	valeur extrême mais compatible avec le phénomène	conserver : elle porte une information

Tableau 7. Trois situations à ne pas confondre.

Une règle statistique repère des candidats, elle ne tranche jamais seule. Sur le jeu Wisconsin, les observations hors bornes sont à 66,7 % malignes alors que les tumeurs malignes ne représentent que 37,3 % de l'échantillon : ces valeurs portent le signal que l'on cherche à prédire, les supprimer serait une erreur. Des tests de cohérence physique (le périmètre doit rester voisin de 2 pi fois le rayon, l'aire de pi fois le rayon au carré) n'ont révélé aucune erreur d'unité grossière : le rapport périmètre sur 2 pi r varie de 0,98 à 1,13.

```
q1, q3 = X.quantile(0.25), X.quantile(0.75)
iqr = q3 - q1
hors = (X < q1 - 1.5 * iqr) | (X > q3 + 1.5 * iqr)
```

Code 4. Repérage de valeurs atypiques par la règle de l'écart interquartile (notebook 03).

POINT D'ATTENTION
Le contrôle de cohérence élimine les erreurs grossières, pas toutes les erreurs : une erreur qui reste cohérente passerait inaperçue.

CHAPITRE 5

Formalisation, prétraitement et partitionnement

Un prétraitement qui apprend quelque chose des données doit être ajusté sur l'entraînement seul. Cette règle conditionne la validité de tout le reste.

5.1 Formaliser le problème d'apprentissage

Dans le cas d'une matrice numérique, les données d'entrée sont notées :

$$X \in \mathbb{R}^{n \times p}, \qquad y \in \{0,1\}^n$$

où n est le nombre d'observations et p le nombre de caractéristiques (ici n = 569 et p = 30). Un modèle est une fonction dont les paramètres sont estimés pendant l'entraînement pour que sa sortie approche y sur les données d'entraînement, dans l'espoir que cette relation **se généralise** à de nouvelles observations. Les hypothèses sont que les observations sont indépendantes et que les données futures ressemblent à celles de l'entraînement.

Tâche	Cible	Exemple biologique
Classification binaire	deux classes	tumeur maligne ou bénigne ; ALL ou AML
Classification multiclasse	plus de deux classes	sous-type moléculaire d'un cancer
Régression	valeur continue	niveau d'expression d'un gène
Non supervisé	aucune étiquette	regroupement d'échantillons, projection (ACP)

Tableau 8. Nature des tâches d'apprentissage. Le développement pratique porte sur la classification supervisée.

5.2 Prétraitement : des transformations qui apprennent

Certaines transformations calculent des paramètres à partir des données : moyenne et écart type pour `StandardScaler`, médiane pour `SimpleImputer`, liste des catégories pour `OneHotEncoder`. scikit-learn distingue `fit`, qui estime ces paramètres sur l'entraînement, et `transform`, qui les applique à l'entraînement comme à la validation et au test. Estimer ces paramètres avec le test introduit une **fuite d'information** : le test n'est plus indépendant du modèle.

Le jeu Wisconsin ne contient ni valeur manquante ni variable catégorielle. Le notebook 03 construit donc une petite table entièrement fictive, explicitement signalée comme telle, pour apprendre ces outils. Le `ColumnTransformer` applique à chaque groupe de colonnes sa propre chaîne de transformations et reconstitue une matrice unique.

```
pretraitement = ColumnTransformer(
    [
        (
            "num",
            Pipeline(
                [
                    ("imputer", SimpleImputer(strategy="median")),
                    ("scaler", StandardScaler()),
                ]
            ),
            numeriques,
        ),
        (
            "cat",
            Pipeline(
                [
                    ("imputer", SimpleImputer(strategy="most_frequent")),
                    (
                        "onehot",
                        OneHotEncoder(handle_unknown="ignore", sparse_output=False),
                    ),
                ]
            ),
            categorielles,
        ),
    ]
)
train_f, test_f = fictif.iloc[:6], fictif.iloc[6:]
matrice_train = pretraitement.fit_transform(train_f)
matrice_test = pretraitement.transform(test_f)
```

Code 5. Imputation, mise à l'échelle et encodage dans un ColumnTransformer (notebook 03, table fictive).

Avec `handle_unknown="ignore"`, une catégorie absente de l'entraînement produit une ligne de zéros au lieu d'une erreur. `LabelEncoder` est réservé à la cible : il attribue des entiers par ordre alphabétique, ce qui suggère une hiérarchie inexistante entre catégories si on l'applique à des variables explicatives. Pour celles-ci, on emploie `OneHotEncoder`.

5.3 Partitionner sans fuite

`train_test_split` découpe les données. Trois arguments comptent : `test_size`, `random_state` qui rend le tirage reproductible, et `stratify=y` qui conserve les proportions de classes dans chaque partie.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED
)
```

Code 6. Partitionnement stratifié et reproductible (notebook 03). Le notebook vérifie ensuite que les index sont disjoints.

Le notebook mesure l'effet d'une erreur d'ajustement : la moyenne de *mean area* apprise sur l'entraînement vaut 659,58 contre 654,89 sur toutes les données. L'écart maximal sur les 30 colonnes atteint 0,049 écart type. L'effet est modeste ici, car l'échantillon est de taille raisonnable ; il est bien plus marqué lorsqu'une sélection supervisée de variables précède la validation, comme le montre le chapitre 10. La solution structurelle est de ne jamais appeler fit à la main, mais d'enfermer les transformations dans un `Pipeline`.

À RETENIR
Le jeu de test est réservé à l'évaluation finale. Il ne sert ni à choisir un modèle, ni à régler un hyperparamètre, ni à décider d'une transformation. Dans les notebooks, il est renommé `X_test_reserve` tant qu'il ne doit pas être utilisé.

5.4 Observations groupées

Si plusieurs observations proviennent du même patient, individu, centre, famille ou lot expérimental, elles sont corrélées. Un partitionnement aléatoire place alors des observations d'un même groupe de part et d'autre de la frontière : le modèle peut reconnaître le groupe au lieu d'apprendre la biologie. On utilise dans ce cas `GroupKFold`, qui garde chaque groupe d'un seul côté. Le chapitre 8 en donne une démonstration chiffrée sur données simulées.

CHAPITRE 6

Modélisation et comparaison algorithmique

Un modèle de référence fixe le niveau minimal à dépasser. Sans lui, aucun score ne se lit.

6.1 Le modèle de référence

`DummyClassifier` ignore les caractéristiques et prédit la classe majoritaire. Sur le jeu de validation, son accuracy est de 0,623 mais son rappel est nul : il ne détecte aucune tumeur maligne. Son AUC vaut exactement 0,5. C'est l'illustration la plus simple de l'insuffisance de l'accuracy.

6.2 Quatre algorithmes et leurs propriétés

Algorithme	Principe	Hyperparamètres	Avantages	Limites
Régression logistique	combinaison linéaire des variables, transformée en probabilité par la sigmoïde	C (inverse de la régularisation)	rapide, coefficients lisibles, bonne référence	frontière linéaire, sensible aux variables corrélées
Arbre de décision	suite de questions binaires qui partitionnent l'espace	max_depth, min_samples_leaf	lisible, sans mise à l'échelle	surapprend, instable
Forêt aléatoire	moyenne de nombreux arbres sur échantillons bootstrap et sous-ensembles de variables	n_estimators, max_depth, max_features	robuste, non-linéaire	moins lisible, coûteuse

Tableau 9. Propriétés des algorithmes retenus. Aucun n'a été ajouté pour allonger le programme.

Paramètres et hyperparamètres. Les paramètres sont appris pendant l'entraînement : coefficients de la régression, variables et seuils des nœuds d'un arbre. Les hyperparamètres sont fixés avant et pilotent la complexité. On ne peut pas les estimer sur l'entraînement seul sans surajustement, on les choisit par validation (chapitre 8). Une mise à l'échelle est utile à la régression logistique, dont l'optimisation et la

régularisation dépendent de l'échelle, et inutile aux arbres, dont les découpages sont invariants par transformation monotone.

6.3 Assembler des pipelines

```
modeles = {
    "Référence (classe majoritaire)": DummyClassifier(strategy="most_frequent"),
    "Régression logistique": Pipeline(
        [("scaler", StandardScaler()), ("modele", LogisticRegression(max_iter=1000))]
    ),
    "Arbre de décision": make_pipeline(DecisionTreeClassifier(random_state=SEED)),
    "Forêt aléatoire": make_pipeline(
        RandomForestClassifier(n_estimators=300, random_state=SEED)
    ),
}
```

Code 7. Quatre pipelines comparés (notebook 04). Le scaler n'est présent que pour la régression logistique.

Un `Pipeline` enchaîne transformations et estimateur sous une interface unique. Lors de `fit`, chaque étape est ajustée sur les données reçues. Lors de `predict`, les étapes déjà ajustées sont appliquées sans être réajustées. Intégré à une validation croisée, le pipeline est réajusté à chaque pli sur son seul entraînement : la fuite est exclue par construction.

6.4 Résultats sur un jeu de validation

Modèle	Accuracy	Précision	Rappel	F1	AUC
Référence (classe majoritaire)	0,623	0,000	0,000	0,000	0,500
Régression logistique	0,974	0,955	0,977	0,966	0,996
Arbre de décision	0,903	0,848	0,907	0,876	0,904
Forêt aléatoire	0,974	0,976	0,954	0,965	0,992

Tableau 10. Performances sur le jeu de validation (114 observations, classe positive : maligne). Le jeu de test n'a pas été consulté.

La régression logistique et la forêt aléatoire sont presque indiscernables, l'arbre non contraint est nettement en retrait : il atteint 1,000 d'accuracy sur l'entraînement et 0,904 sur la validation, ce qui mesure son surapprentissage. Limiter sa profondeur est une forme de régularisation. Avec 114 observations, une seule erreur modifie l'accuracy d'environ 0,9 point : une différence d'un à deux points ne permet pas de conclure, ce qui motive la validation croisée.

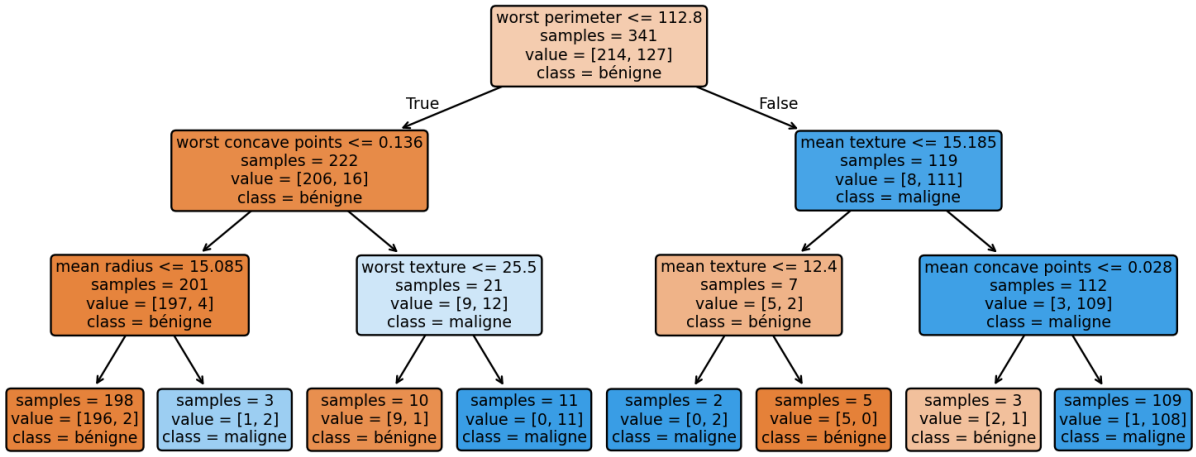


Figure 5. Arbre de profondeur 3 ajusté sur l'entraînement. Chaque chemin de la racine à une feuille est une règle lisible.

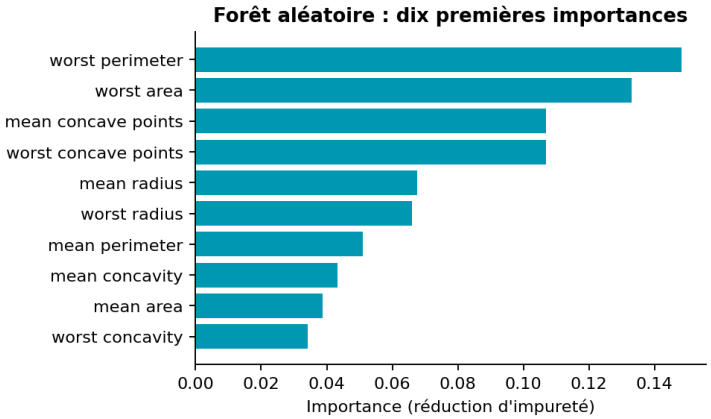


Figure 6. Importances de la forêt aléatoire. Elles se répartissent entre variables corrélées.

POINT D'ATTENTION
L'importance d'une variable est une propriété du modèle ajusté, pas du phénomène biologique. Lorsque rayon, périmètre et aire sont corrélés, l'importance se divise entre eux : aucune n'apparaît aussi forte qu'elle le serait seule.

CHAPITRE 7

Évaluation expérimentale

Une métrique n'a de sens que reliée à la question scientifique. Elle doit être choisie avant de voir les résultats.

7.1 Matrice de confusion : un scénario biologique

La classe positive est *tumeur maligne*. Un **vrai positif** est une tumeur maligne prédite maligne : la patiente est orientée vers la prise en charge. Un **faux négatif** est une tumeur maligne prédite bénigne : le diagnostic est manqué et le retard de traitement peut être grave. Un **faux positif** est une tumeur bénigne prédite maligne : examens complémentaires inutiles et anxiété, sans danger vital. Un **vrai négatif** est une tumeur bénigne prédite bénigne. Les deux erreurs n'ont pas le même coût : en dépistage, on accepte plus volontiers des faux positifs que des faux négatifs.

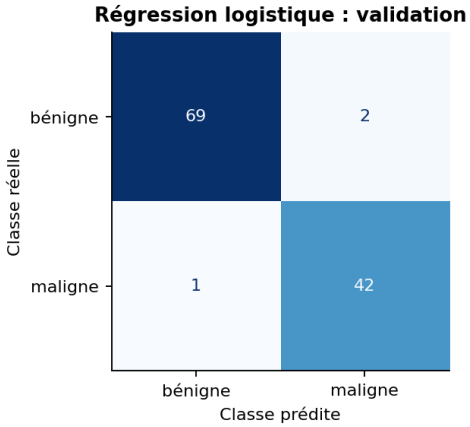


Figure 7. Matrice de confusion de la régression logistique sur la validation : VN = 69, FP = 2, FN = 1, VP = 42.

7.2 Définitions des métriques

$$\text{accuracy} = \frac{VP + VN}{n}, \quad \text{précision} = \frac{VP}{VP + FP}, \quad \text{rappel} = \frac{VP}{VP + FN}$$

$$F_1 = \frac{2 \cdot \text{précision} \cdot \text{rappel}}{\text{précision} + \text{rappel}}$$

L'**AUC** est l'aire sous la courbe ROC. C'est la probabilité qu'une observation positive tirée au hasard reçoive un score plus élevé qu'une observation négative tirée au hasard. Elle mesure la **discrimination**, indépendamment du seuil de décision. Dans scikit-learn, ces mesures correspondent à `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `roc_auc_score`, `confusion_matrix` et `classification_report`.

```
def evaluer(modele, X_a, y_a):
    """Retourne les cinq métriques d'un modèle déjà ajusté. La classe positive est 1 (maligne)."""
    pred = modele.predict(X_a)
    score = modele.predict_proba(X_a)[:, 1]
    return {
        "accuracy": accuracy_score(y_a, pred),
        "précision": precision_score(y_a, pred, zero_division=0),
        "rappel": recall_score(y_a, pred, zero_division=0),
        "F1": f1_score(y_a, pred, zero_division=0),
        "AUC": roc_auc_score(y_a, score),
    }
```

Code 8. Calcul des cinq métriques avec la classe maligne comme classe positive (notebook 04).

7.3 Choisir et justifier ses métriques

- **Accuracy.** Insuffisante dès que les classes sont déséquilibrées : le modèle de référence atteint 0,62 en ne détectant rien. Sur le jeu de Golub, il atteindrait 0,65.
- **Rappel.** Mesure la fraction de tumeurs malignes détectées. Il peut être maximisé trivialement en prédisant toujours maligne, il doit donc être lu avec la précision.
- **Précision.** Mesure la fiabilité d'une alerte. Elle chute quand les faux positifs augmentent.
- **F1.** Résume précision et rappel pour un seuil donné.
- **AUC.** Compare des modèles indépendamment du seuil, mais ne dit pas quel seuil retenir.

Trois notions sont à distinguer. La **discrimination** est la capacité à classer les observations selon un score. La **calibration** est la fiabilité des probabilités : une prédiction à 0,9 est calibrée si environ 90 % des observations qui la reçoivent sont réellement positives. La **valeur pratique** d'une prédiction dépend du coût des erreurs, de la prévalence et de l'usage clinique. Un modèle peut bien discriminer et être mal calibré.

POINT D'ATTENTION
Une bonne performance interne n'est pas une preuve de validité clinique. Celle-ci exige des données indépendantes, acquises par d'autres équipes, puis une évaluation prospective.

CHAPITRE 8

Validation croisée, optimisation et évaluation finale

L'optimisation consomme de l'information sur les données : elle doit rester séparée de l'évaluation finale.

8.1 Validation croisée stratifiée

Les données d'entraînement sont découpées en k plis. Pour chaque pli, le modèle est ajusté sur les k-1 autres et évalué sur celui qui reste. La moyenne des k scores estime la performance attendue, leur écart type mesure la **variabilité** liée au découpage. `StratifiedKFold` conserve les proportions de classes, `shuffle=True` avec `random_state` rend le tirage reproductible. Tous les modèles sont évalués avec le même objet de découpage, sur des plis identiques, pour que la comparaison soit appariée.

Modèle	Accuracy	F1	Rappel	AUC
Référence	0,626 ± 0,000	0,000 ± 0,000	0,000 ± 0,000	0,500 ± 0,000
Régression logistique	0,974 ± 0,015	0,964 ± 0,021	0,953 ± 0,040	0,996 ± 0,005
Arbre de décision	0,923 ± 0,033	0,896 ± 0,045	0,894 ± 0,061	0,917 ± 0,037
Forêt aléatoire	0,963 ± 0,015	0,949 ± 0,021	0,935 ± 0,039	0,989 ± 0,007

Tableau 11. Validation croisée à 5 plis sur l'entraînement (455 observations). Moyenne ± écart type entre plis.

Le format moyenne ± écart type rend visible l'incertitude que la validation unique cachait. Même la moyenne dépend de la graine des plis : pour la régression logistique, l'accuracy moyenne varie de 0,969 à 0,978 selon la graine, sur dix graines. Quand l'écart entre deux modèles est du même ordre que cet écart type, on ne peut pas affirmer que l'un est meilleur.

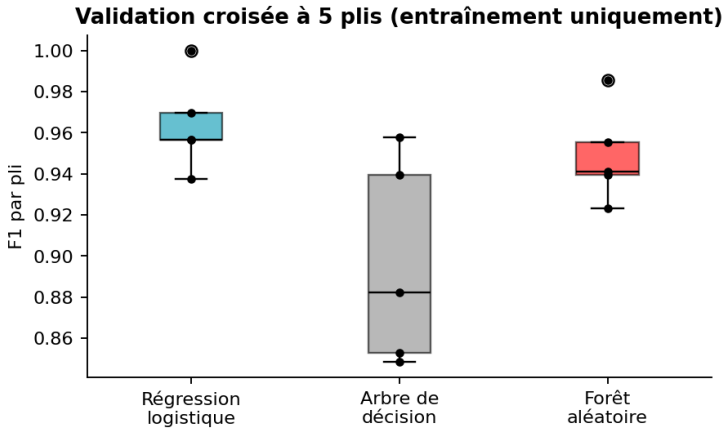


Figure 8. F1 par pli pour trois modèles. Chaque point est un pli de validation.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
scoring = {"accuracy": "accuracy", "recall": "recall", "f1": "f1", "roc_auc": "roc_auc"}
res = cross_validate(pipeline, X_train, y_train, cv=cv, scoring=scoring)
print(res["test_roc_auc"].mean(), res["test_roc_auc"].std())
```

Code 9. Validation croisée avec plusieurs métriques (extrait simplifié du notebook 05).

8.2 Rechercher les hyperparamètres

`GridSearchCV` ajuste le pipeline pour chaque combinaison d'une grille, l'évalue par validation croisée, puis réajuste la meilleure sur tout l'entraînement. Dans un pipeline, les hyperparamètres d'une étape se nomment `etape__parametre`. `RandomizedSearchCV` tire au hasard `n_iter` configurations dans des distributions, ce qui est plus économe lorsque l'espace est grand.

```
pipe_lr = Pipeline(
    [("scaler", StandardScaler()), ("modele", LogisticRegression(max_iter=2000))]
)
grille_lr = {"modele__C": [0.001, 0.01, 0.1, 1, 10, 100]}
gs_lr = GridSearchCV(
    pipe_lr, grille_lr, scoring=scoring_gs, refit="roc_auc", cv=cv
).fit(X_train, y_train)
```

Code 10. Recherche exhaustive du paramètre C de la régression logistique, sélection sur l'AUC (notebook 05).

Choix de la métrique de sélection. L'AUC est retenue pour comparer les configurations parce qu'elle est indépendante du seuil. Le seuil est choisi ensuite, séparément. La recherche retient C = 1 avec une AUC de validation croisée de 0,996 ; l'AUC varie peu autour de l'optimum, plusieurs valeurs de C sont donc pratiquement équivalentes. Pour la forêt, la recherche aléatoire sur 12 configurations retient une AUC de 0,989.

8.3 Validation croisée imbriquée

Le score `best_score_` d'une recherche est optimiste : la meilleure configuration a été choisie parce qu'elle avait le meilleur score sur ces plis. Pour estimer la procédure complète, on imbrique une boucle interne qui optimise et une boucle externe qui évalue. L'AUC imbriquée de la régression logistique est de 0,9945 ± 0,0042, contre 0,9958 sans imbrication. L'écart est faible ici parce que l'espace de recherche est petit et le signal fort ; il serait plus important avec de nombreux hyperparamètres, un petit effectif ou un signal faible.

```
cv_externe = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED + 1)
cv_interne = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED + 2)

recherche_interne = GridSearchCV(pipe_lr, grille_lr, scoring="roc_auc", cv=cv_interne)
auc_imbriquee = cross_val_score(
    recherche_interne, X_train, y_train, cv=cv_externe, scoring="roc_auc"
)
```

Code 11. Validation croisée imbriquée (notebook 05).

8.4 Observations groupées : démonstration

Sur des données **entièrement simulées**, construites pour que l'étiquette n'ait aucun lien avec les mesures (chaque patient a une signature propre, indépendante de sa classe), un découpage aléatoire annonce une accuracy de 0,996. Avec `GroupKFold`, qui sépare les patients, elle tombe à 0,546, au niveau du hasard (la classe majoritaire représente 0,60). Toute performance supérieure au hasard dans la première procédure ne vient que de la reconnaissance du patient.

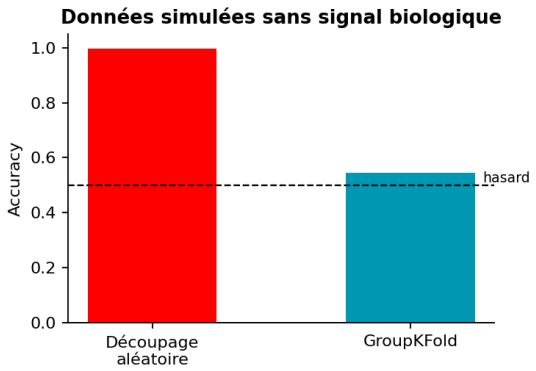


Figure 9. Accuracy sur données simulées sans signal biologique, selon le découpage.

L'expérience est une construction pédagogique, mais le mécanisme est réel. La copie du jeu Wisconsin utilisée ne contient pas d'identifiant patient : l'indépendance des observations ne peut pas y être vérifiée, ce qui constitue une limite.

8.5 Règle de sélection, seuil et évaluation unique

La règle est écrite avant de regarder le test : on retient le modèle de plus haute AUC en validation croisée, et le plus simple si l'écart est inférieur à l'écart type du meilleur. Avec 0,9958 ± 0,0047 pour la régression logistique et 0,9885 ± 0,0056 pour la forêt, l'écart est inférieur à l'écart type : la régression logistique est retenue. Le seuil de décision est ensuite choisi sur des prédictions de validation croisée de l'entraînement : on cherche le seuil le plus élevé qui garantit un rappel d'au moins 0,97, exigence posée a priori. Le seuil obtenu est 0,315, avec un rappel de validation croisée de 0,971 et une précision de 0,954.

Le modèle est alors figé et le jeu de test, de 114 observations dont 42 malignes, est utilisé une fois.

Métrique	Seuil 0,5	Seuil 0,315	IC95 % bootstrap (seuil 0,5)
Accuracy	0,965	0,974	[0,930 ; 0,991]
Précision	0,975	0,976	
Rappel	0,929	0,952	[0,841 ; 1,000]
F1	0,951	0,964	
AUC	0,996	0,996	[0,986 ; 1,000]
Faux négatifs, faux positifs	3 et 1	2 et 1	

Tableau 12. Évaluation finale de la régression logistique sur le jeu de test (114 observations, 42 malignes), effectuée une fois.

Abaisser le seuil de 0,5 à 0,315 évite un faux négatif sans créer de faux positif supplémentaire. Avec 42 tumeurs malignes, un faux négatif de plus ou de moins change le rappel de plus de deux points, d'où des intervalles larges : l'IC95 % bootstrap du rappel descend jusqu'à 0,84. Le bootstrap rééchantillonne le jeu de test avec remise et recalcule la métrique, le modèle n'est pas réajusté.

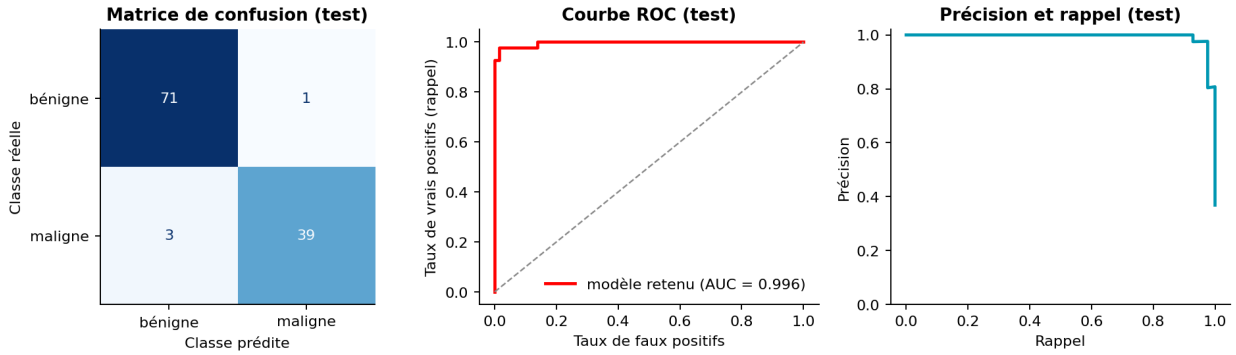


Figure 10. Évaluation finale : matrice de confusion, courbe ROC et courbe précision rappel sur le jeu de test.

La calibration est examinée sur 5 groupes. Avec un test de cette taille, la courbe est très approximative : elle sert à repérer un défaut grossier, non à certifier une calibration. Si les résultats avaient déçu, il n'aurait pas été permis de modifier le modèle en regardant le test : cela l'aurait transformé en jeu de validation.

8.6 Sauvegarder, recharger, prédire

Sauvegarder le pipeline entier, et non le seul estimateur, garantit que les mêmes transformations sont appliquées aux futures données. Le fichier sérialisé n'est fiable qu'avec la même version de scikit-learn, et il ne faut jamais charger un fichier de provenance inconnue, car la désérialisation peut exécuter du code. Le notebook vérifie que les prédictions du modèle rechargé sont identiques sur les 114 observations et qu'une entrée dont une colonne manque est refusée avec une erreur explicite.

```
recharge = joblib.load(chemin)
assert np.array_equal(recharge.predict(X_test), modele_final.predict(X_test))
assert np.allclose(recharge.predict_proba(X_test), modele_final.predict_proba(X_test))
```

Code 12. Rechargement du pipeline et vérification de l'identité des prédictions (notebook 05).

POINT D'ATTENTION
L'optimisation d'hyperparamètres sur un petit jeu de données ne garantit ni la robustesse à un autre centre, un autre instrument ou une autre population, ni la validité biologique.

CHAPITRE 9

Introduction opérationnelle au deep learning

Un réseau de neurones est une composition de fonctions simples dont on ajuste les poids pour réduire une perte.

9.1 Quand le deep learning se justifie

Il excelle lorsque la structure des données est riche et le volume suffisant : images d'histopathologie ou de microscopie (réseaux convolutifs), séquences d'ADN, d'ARN ou de protéines, grands volumes de dizaines de milliers d'observations, données multimodales. Il est rarement avantageux sur un tableau de quelques centaines de lignes : plus de paramètres que de données, hyperparamètres nombreux, résultats plus variables, interprétation plus difficile. Le notebook 06 poursuit donc une visée de compréhension des mécanismes, non de performance.

9.2 Neurone, perte et gradient

Un **tenseur** est un tableau multidimensionnel de forme et de type définis, de rang 0 pour un scalaire, 1 pour un vecteur, 2 pour une matrice. Un neurone calcule une somme pondérée de ses entrées, ajoute un biais, puis applique une fonction d'activation :

$$z = w^\top x + b, \qquad \hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

Les poids w et le biais b sont les paramètres appris. La sigmoïde transforme un réel en valeur comprise entre 0 et 1, interprétable comme une probabilité. Sans activation non linéaire, un empilement de couches équivaudrait à une seule transformation linéaire : la fonction ReLU, qui vaut max(0, z), donne au réseau sa capacité à représenter des relations non linéaires. La **perte** mesure l'écart entre prédiction et vérité. Pour une classification binaire, c'est l'entropie croisée :

$$L = -\frac{1}{n}\sum_{i=1}^{n}\left[\, y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\,\right]$$

Elle pénalise lourdement une prédiction confiante et fausse : prédire 0,01 pour une tumeur maligne coûte 4,605, contre 0,916 pour une prédiction à 0,4. Le **gradient** de la perte par rapport à un poids indique dans quel sens et avec quelle intensité la perte varie quand ce poids change. On déplace chaque poids dans la direction opposée, avec un pas appelé taux d'apprentissage, hyperparamètre fondamental. La **rétropropagation** applique la règle de dérivation des fonctions composées de la sortie vers l'entrée, ce qui fournit le gradient de chaque poids de chaque couche. TensorFlow le calcule avec `GradientTape`.

```
w_tf = tf.Variable(rng.normal(size=(3,)).astype("float32"))
b_tf = tf.Variable(0.0)
with tf.GradientTape() as tape:
    p = tf.sigmoid(tf.linalg.matvec(Xt, w_tf) + b_tf)
    perte = keras.losses.BinaryCrossentropy()(yt, p)
g_w, g_b = tape.gradient(perte, [w_tf, b_tf])
```

Code 13. Calcul automatique du gradient d'un neurone, comparé ensuite au gradient analytique (notebook 06).

Le notebook vérifie que le gradient automatique coïncide avec la formule analytique, puis écrit une boucle de descente de gradient où la perte passe de 0,693 à 0,053 en 200 étapes sur deux nuages fictifs. Un **lot** (mini-batch) est un sous-ensemble d'observations utilisé à chaque étape, une **époque** est un passage complet sur les données d'entraînement.

9.3 Construire, entraîner, surveiller

Le protocole est compatible avec celui du chapitre 8 : mêmes graines, même partition entraînement et test, même classe positive. Un jeu de validation de 20 % est extrait de l'entraînement pour l'arrêt anticipé, et le `StandardScaler` n'est ajusté que sur les données d'ajustement. L'architecture compte deux couches cachées (16 puis 8 neurones, ReLU), un `Dropout` de 0,2 et une sortie sigmoïde, soit 641 paramètres pour 364 observations d'ajustement. Le dropout met à zéro au hasard 20 % des activations pendant l'entraînement seulement, ce qui empêche de dépendre de neurones particuliers.

```
def construire_reseau(unites=(16, 8), dropout=0.2, lr=1e-3):
    modele = keras.Sequential(
        [layers.Input(shape=(30,))]
        + [layers.Dense(u, activation="relu") for u in unites[:1]]
        + ([layers.Dropout(dropout)] if dropout > 0 else [])
        + [layers.Dense(u, activation="relu") for u in unites[1:]]
        + [layers.Dense(1, activation="sigmoid")]
    )
    modele.compile(
        optimizer=keras.optimizers.Adam(learning_rate=lr),
        loss="binary_crossentropy",
        metrics=["accuracy", keras.metrics.AUC(name="auc")],
    )
    return modele
```

Code 14. Définition et compilation du réseau dense : Adam, entropie croisée binaire, AUC suivie (notebook 06).

```
arret = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=20, restore_best_weights=True
)
historique = reseau.fit(
    A_fit,
    b_fit,
    validation_data=(A_val, b_val),
    epochs=300,
    batch_size=32,
    callbacks=[arret],
    verbose=0,
)
```

Code 15. Entraînement avec arrêt anticipé sur la perte de validation (notebook 06).

9.4 Diagnostiquer le surapprentissage

Les courbes de perte d'entraînement et de validation sont l'instrument principal. Pour provoquer le phénomène, un réseau volumineux de 37121 paramètres est entraîné 200 époques sans dropout ni arrêt anticipé : sa perte de validation atteint un minimum de 0,058 à l'époque 7, puis remonte à 0,178 à l'époque 200 alors que sa perte d'entraînement tend vers zéro. Le réseau régularisé s'arrête à l'époque 209 sur 229 exécutées, avec une AUC de validation de 0,983.

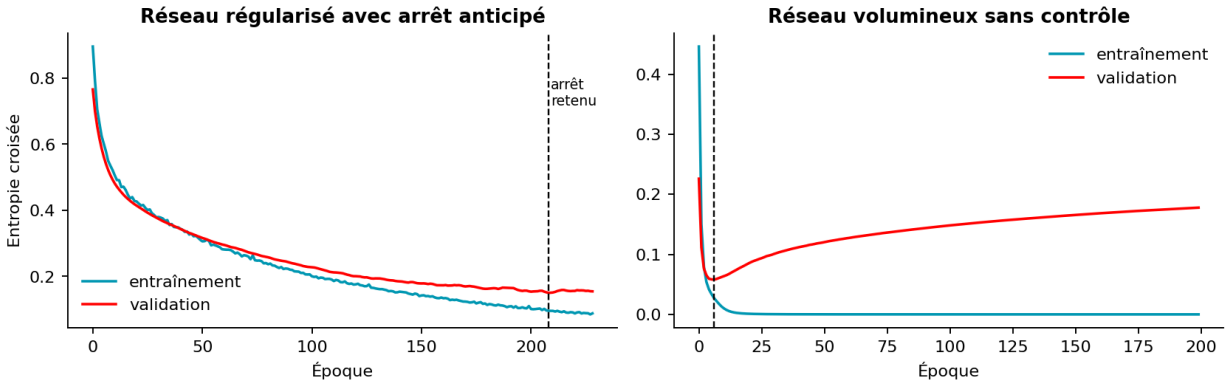


Figure 11. Courbes de perte. À gauche, réseau régularisé avec arrêt anticipé. À droite, réseau volumineux sans contrôle : la perte de validation remonte après un minimum précoce.

Variabilité due à la graine. Un réseau est initialisé au hasard. Sur cinq graines et sur le jeu de validation, l'AUC vaut en moyenne 0,9917 avec un écart type de 0,0032 (minimum 0,9856, maximum 0,9943). La graine 42 du réseau principal a été fixée avant tout résultat et n'a pas été remplacée : elle se révèle plus lente que quatre des cinq autres graines. On la rapporte telle quelle.

9.5 Comparaison dans des conditions compatibles

Le réseau est ajusté sur 80 % de l'entraînement, les 20 % restants servant à l'arrêt anticipé. La régression logistique de référence est donc ajustée sur ce même sous-ensemble, avec le même scaler et C = 1 (choisi par validation croisée au chapitre 8, indépendamment du test). Les deux modèles voient les mêmes données et sont évalués sur le même test. Comparer ce réseau au modèle du chapitre 8, ajusté sur 100 % de l'entraînement, ne serait pas équitable.

Métrique	Réseau dense	Régression logistique
Accuracy	0,974	0,983
Précision	0,976	0,976
Rappel	0,952	0,976
F1	0,964	0,976
AUC	0,987	0,996

Tableau 13. Jeu de test, seuil 0,5, mêmes données d'ajustement pour les deux modèles.

Avec 114 observations de test, quelques échantillons suffisent à inverser un classement : les deux modèles sont statistiquement indiscernables. On ne conclut pas que l'un est supérieur, on constate que le réseau, plus complexe, n'apporte pas de gain démontrable, ce qui est le résultat attendu pour un tableau de cette taille.

CHAPITRE 10

Projet intégrateur : expression génique et leucémies

La compétence finale se démontre sur un jeu inconnu : on applique la démarche complète à des données où p est près de cent fois supérieur à n.

10.1 Contexte et contrôles d'entrée

Golub et collaborateurs ont profilé 72 échantillons de leucémie aiguë sur puces Affymetrix : 47 leucémies lymphoblastiques (ALL) et 25 myéloïdes (AML), 7129 sondes. La classe positive est AML. Le notebook télécharge le fichier (1 191 760 octets, SHA-256 affiché pour détecter toute altération), puis vérifie par assertions les dimensions, l'unicité des identifiants et des sondes, l'absence de valeur manquante, les effectifs de classes et l'absence de la cible parmi les caractéristiques.

```
X = g["x"].to_pandas()
X.index = X.index.astype(str)
X.columns = X.columns.astype(str)
X.index.name, X.columns.name = None, None
etiquette = pd.Series(np.asarray(g["y"]).astype(str), index=X.index, name="classe")
y = (
    (etiquette == "AML").astype(int).rename("AML")
)  # classe positive : AML (1) ; ALL (0)
```

Code 16. Construction de la matrice d'expression et de la cible à partir du fichier (notebook 07).

Le jeu de test (18 échantillons dont 6 AML) est mis de côté **avant** toute exploration orientée par les classes : l'analyse en composantes principales n'utilise que l'entraînement. Les valeurs sont des intensités dont 28,8 % sont négatives, ce qui exclut une transformation logarithmique directe, et les écarts types des sondes s'étalent de 25 à plus de 16 000 : une mise à l'échelle par sonde est indispensable. Une règle qui prédirait toujours ALL atteindrait environ 0,65 d'accuracy.

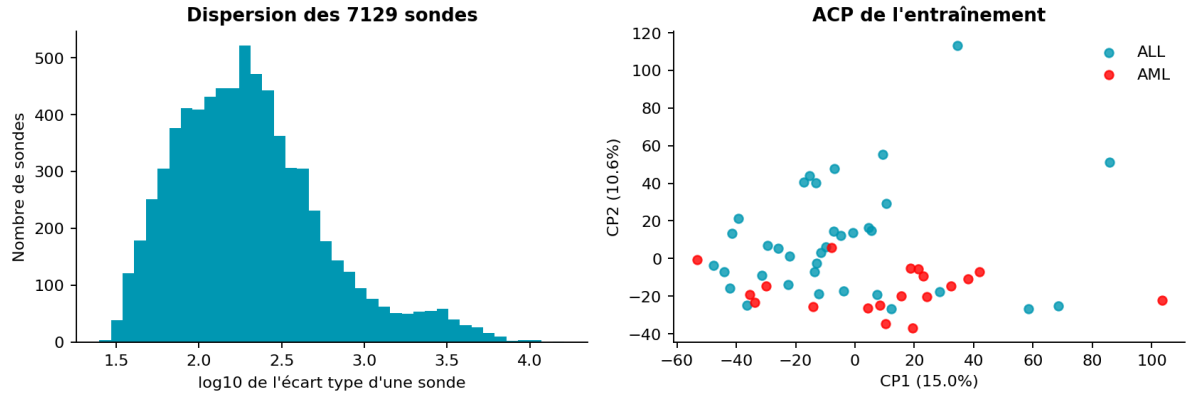


Figure 12. Dispersion des 7129 sondes et ACP de l'entraînement. Les deux premières composantes expliquent 15,0 % et 10,6 % de la variance.

10.2 La fuite par sélection de gènes, démontrée

La sélection de gènes utilise les étiquettes. Exécutée sur tout l'entraînement avant la validation croisée, elle choisit des gènes en voyant aussi les échantillons de validation. Pour le montrer, on permute aléatoirement les étiquettes : il n'existe alors aucune relation entre expression et classe, et la seule performance honnête est celle du hasard, une AUC de 0,5.

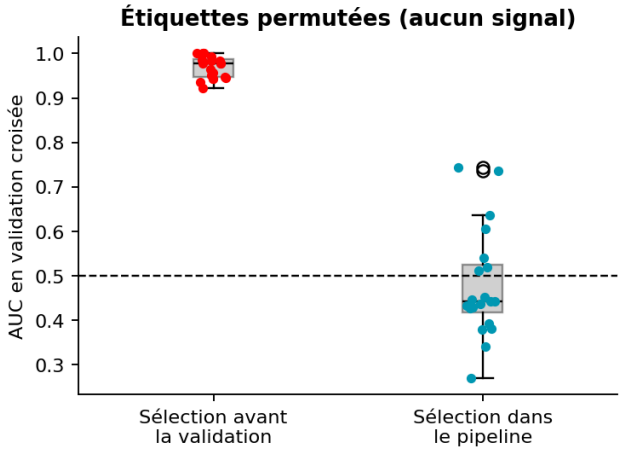


Figure 13. AUC en validation croisée sur 20 permutations des étiquettes, selon que la sélection de gènes est faite avant la validation ou à l'intérieur du pipeline.

Sur 20 permutations, la procédure fautive annonce une AUC moyenne de 0,969 (de 0,921 à 1,000) sur des données sans aucun signal, alors que la procédure correcte reste à 0,478. Avec les vraies étiquettes, l'écart est plus faible : 1,000 contre 0,964, puisqu'un vrai signal existe. Cette expérience rappelle les travaux d'Ambroise et McLachlan sur le biais de sélection en génomique.

```
def pipeline_selection(k=50, C=0.1):
    return Pipeline(
        [
            ("scaler", StandardScaler()),
            ("selection", SelectKBest(f_classif, k=k)),
            ("modele", LogisticRegression(C=C, max_iter=2000)),
        ]
    )
```

Code 17. La sélection de gènes placée dans le pipeline est recalculée à chaque pli (notebook 07).

À RETENIR
Toute étape qui utilise l'étiquette, c'est-à-dire sélection de variables, réduction supervisée ou choix de seuil, appartient au pipeline.

10.3 Comparaison par validation croisée imbriquée

Avec 54 échantillons d'entraînement, un seul découpage serait très bruité. Chaque procédure complète, prétraitement, sélection et réglage compris, est évaluée par validation croisée imbriquée : boucle externe répétée à 4 plis et 3 répétitions, boucle interne à 3 plis.

Candidat	AUC	Rappel AML	Balanced accuracy
Référence	0,500 ± 0,000	0,000 ± 0,000	0,500 ± 0,000
Logistique, tous les gènes	0,974 ± 0,039	0,829 ± 0,198	0,886 ± 0,101
Sélection de gènes + logistique	0,972 ± 0,044	0,867 ± 0,189	0,923 ± 0,095
Forêt aléatoire	0,989 ± 0,024	0,854 ± 0,221	0,927 ± 0,111

Tableau 14. Validation croisée imbriquée sur l'entraînement (54 échantillons). Les ± indiquent la dispersion entre plis externes, pas un intervalle de confiance.

Les écarts types sont importants, avec des plis externes d'environ 13 échantillons qui partagent des données d'entraînement. La règle de sélection écrite à l'avance retient le candidat de plus haute AUC, et parmi ceux à moins d'un écart type de lui, le plus parcimonieux. Les trois candidats apprenants sont ici dans cet intervalle : la sélection de gènes avec régression logistique est retenue. Le réglage sur tout l'entraînement retient k = 500 sondes et C = 0.1. La parcimonie n'a donc servi qu'à départager des candidats équivalents, elle ne garantit pas un modèle à quelques gènes.

10.4 Évaluation finale et lecture prudente

Sur les 18 échantillons de test, le modèle classe correctement les 18 : 6 AML détectées sur 6, 12 ALL correctes sur 12, aucune erreur. La référence atteindrait 0,667. Ce résultat ne doit pas être surinterprété. Un bootstrap serait dégénéré, puisque tous les rééchantillons sont parfaits ; on utilise donc des intervalles exacts de Clopper et Pearson. L'accuracy a pour intervalle à 95 % [0,815 ; 1,000] et le rappel de la classe AML, 6 sur 6, [0,541 ; 1,000]. La validation croisée imbriquée donnait un rappel moyen de 0,867 avec un écart type de 0,189 : c'est cette estimation, plus riche, qui décrit la performance attendue, non le score du test.

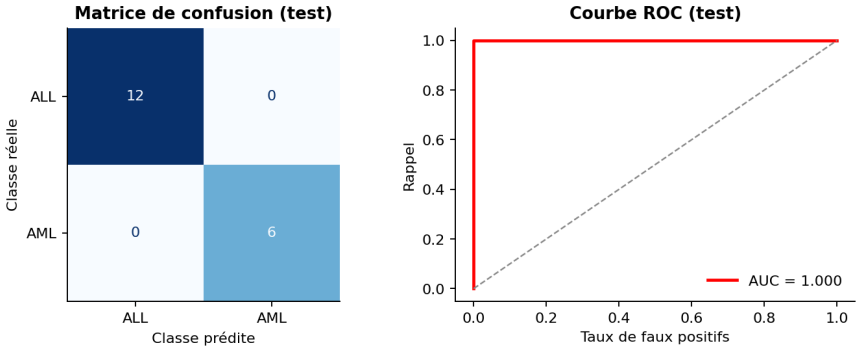


Figure 14. Matrice de confusion et courbe ROC sur les 18 échantillons de test.

10.5 Stabilité de la sélection et limites

Les sondes retenues sont des candidats statistiques, non des gènes validés : le fichier ne contient pas l'annotation de la plateforme, donc aucune identité de gène n'est affirmée. On évalue plutôt la stabilité de la sélection sur 20 sous-échantillons de 80 % de l'entraînement : l'indice de Jaccard moyen entre listes de 20 sondes est de 0,452, et 8 sondes figurent dans au moins 18 listes sur 20. Près de la moitié de la liste change donc d'un sous-échantillon à l'autre, tandis qu'un petit noyau revient presque toujours, ce qui est typique de données où de nombreuses sondes sont corrélées et informatives.

Limite	Conséquence
72 échantillons, 18 en test	intervalles larges ; classement fin des modèles impossible
Tâche relativement séparable	la performance ne dit rien de tâches plus subtiles (sous-types, pronostic)
Métadonnées absentes (centre, tissu, sexe)	un effet de lot confondu avec la classe ne peut être détecté ici
Aucune cohorte indépendante	performance interne, pas validité externe
Sondes sélectionnées	association statistique, pas rôle causal démontré
Source de données	miroir public dont la licence n'a pas été vérifiée

Tableau 15. Limites du projet intégrateur et leurs conséquences.

CHAPITRE 11

Reproductibilité, interprétation scientifique et suite

Un résultat n'a de valeur que s'il peut être reproduit et correctement interprété.

11.1 Procédure de validation du code

Chaque notebook suit six étapes : contrôle de l'environnement, validation des entrées par assertions, exécution progressive depuis un état propre, contrôle des résultats (dimensions, types, cohérence des partitions, forme des prédictions), reproductibilité par graines fixées, et validation finale par réexécution complète. Les notebooks 01 à 05 et 07 ont été réexécutés de bout en bout depuis un état propre sans erreur ; le notebook 06 a été exécuté intégralement avec succès. Les messages techniques de TensorFlow, par exemple sur les opérations oneDNN, sont des informations et non des erreurs, mais ils signalent que les résultats numériques peuvent différer légèrement entre processeurs.

Vérification demandée	Où elle est faite
La cible n'est pas parmi les caractéristiques	assertions des notebooks 02, 03, 04, 07
Aucun prétraitement n'est ajusté sur le test	pipelines des notebooks 04, 05, 07 ; scaler du notebook 06 ajusté sur l'ajustement
Le test ne sert pas à choisir les hyperparamètres	recherches sur l'entraînement seul, test utilisé en fin de notebooks 05, 06, 07
Les métriques correspondent à la tâche	classe positive explicite, AUC et rappel suivis
Les résultats viennent de l'exécution réelle	journaux JSON écrits par les notebooks et lus pour ce livre
Le modèle sauvegardé est rechargeable	assertions d'identité des prédictions (notebooks 05, 06, 07)

Tableau 16. Contrôles méthodologiques du projet et leur localisation.

11.2 Trois niveaux d'interprétation

Niveau	Question	Ce qui l'établit	Ce qui ne suffit pas
Performance prédictive	le modèle prédit-il bien des données jamais vues ?	évaluation sur un test indépendant, avec incertitude	un score élevé sur un petit test
Association statistique	une variable est-elle liée à la cible ?	tests, stabilité de la sélection, réplication	l'importance d'une variable dans un modèle
Conclusion biologique	un mécanisme est-il impliqué ?	annotation, expériences, cohortes indépendantes	toute performance prédictive seule

Tableau 17. Distinguer prédiction, association et mécanisme.

11.3 Limites de ce dispositif

- Les serveurs UCI et GEO étant inaccessibles lors de la conception, le second jeu de données provient d'un miroir GitHub dont la licence n'est pas vérifiée. Une extension propose de reproduire la démarche sur une série GEO officielle.
- Les notebooks ont été validés sur un seul environnement (Python 3.12). Leur exécution dans Google Colab n'a pas pu être testée depuis cet environnement.
- Les liens d'ouverture dans Colab pointent vers le dépôt prévu, qui n'est pas encore publié.
- Le jeu Wisconsin ne fournit pas d'identifiant patient dans la copie utilisée, l'hypothèse d'indépendance n'est pas vérifiable.
- Aucune des performances présentées ne constitue une validation clinique.

11.4 Pour aller plus loin

Trois pistes prolongent le parcours : répliquer le projet sur une série d'expression issue de NCBI GEO en documentant provenance, licence et plateforme ; aborder la régression et l'apprentissage non supervisé avec les mêmes exigences de protocole ; étudier les réseaux convolutifs sur des séquences ou des images, domaines où le deep learning est réellement justifié.

ANNEXE A

Glossaire

Terme	Définition
Observation	une ligne du tableau de données : un échantillon, un patient, une cellule.
Caractéristique (feature)	une colonne d'entrée du modèle, représentation numérique d'une mesure.
Étiquette (cible)	la valeur que le modèle doit prédire. En classification, la classe.
Paramètre	valeur apprise pendant l'entraînement (coefficient, poids, seuil de nœud).
Hyperparamètre	valeur fixée avant l'entraînement, qui pilote la complexité (C, profondeur, taux d'apprentissage).
Pipeline	enchaînement de transformations et d'un estimateur, ajusté d'un seul bloc.
Fuite d'information	utilisation, lors de l'ajustement, d'informations issues des données de validation ou de test.
Modèle de référence (baseline)	modèle trivial qui fixe le niveau minimal à dépasser.
Surapprentissage	le modèle mémorise l'entraînement et généralise mal : grand écart entre entraînement et validation.
Stratification	découpage qui conserve les proportions de classes dans chaque partie.
Validation croisée	alternance de plis d'entraînement et de validation pour estimer une performance et sa variabilité.
Validation croisée imbriquée	validation croisée externe qui évalue une procédure dont l'optimisation est faite dans une boucle interne.
Précision, rappel	fiabilité des alertes positives, fraction des positifs réels détectés.
AUC	probabilité qu'un positif reçoive un score supérieur à celui d'un négatif ; mesure de discrimination.
Calibration	accord entre les probabilités prédites et les fréquences observées.
Époque, lot	passage complet sur l'entraînement ; sous-ensemble d'observations utilisé à une étape de mise à jour.
Gradient, rétropropagation	sensibilité de la perte à chaque poids ; calcul de ce gradient de la sortie vers l'entrée.
Dropout, arrêt anticipé	deux régularisations : désactivation aléatoire de neurones, arrêt quand la perte de validation cesse de baisser.
Bootstrap	rééchantillonnage avec remise utilisé pour estimer l'incertitude d'une métrique.
Intervalle de Clopper et Pearson	intervalle de confiance exact d'une proportion, adapté aux petits effectifs.

Tableau 18. Termes techniques employés dans le parcours.

ANNEXE B

Liste de contrôle avant d'annoncer un résultat

N°	Question à se poser
1	La classe positive est-elle explicite et correspond-elle à la question scientifique ?
2	La cible est-elle absente de la matrice de caractéristiques ?
3	Un modèle de référence a-t-il été évalué ?
4	Les observations sont-elles indépendantes ? Existe-t-il des groupes (patients, centres, lots) ?
5	Le jeu de test a-t-il été isolé avant toute exploration orientée par la cible ?
6	Tout ce qui apprend des données, y compris la sélection de variables, est-il dans le pipeline ?
7	Les hyperparamètres ont-ils été choisis sans le jeu de test ?
8	La règle de sélection du modèle a-t-elle été écrite avant de voir le test ?
9	Les métriques sont-elles adaptées au déséquilibre et au coût des erreurs ?
10	Les performances sont-elles rapportées avec leur incertitude (écart type, intervalle) ?
11	Les graines sont-elles fixées et les versions des bibliothèques consignées ?
12	Le modèle sauvegardé a-t-il été rechargé et vérifié, avec validation des entrées ?
13	Les limites (effectif, provenance, absence de validation externe) sont-elles énoncées ?
14	L'interprétation distingue-t-elle prédiction, association et mécanisme biologique ?

Tableau 19. Quatorze questions qui résument le protocole du parcours.

RÉFÉRENCES

Bibliographie

Documentations officielles. NumPy, https://numpy.org/doc/stable/ ; pandas, https://pandas.pydata.org/docs/ ; Matplotlib, https://matplotlib.org/stable/ ; scikit-learn, https://scikit-learn.org/stable/ ; TensorFlow, https://www.tensorflow.org/learn ; Google Colab, https://colab.research.google.com/.

Wolberg W. H., Street W. N., Mangasarian O. L. *Breast Cancer Wisconsin (Diagnostic)*. UCI Machine Learning Repository, 1995. https://doi.org/10.24432/C5DW2B. Licence CC BY 4.0.

Golub T. R., Slonim D. K., Tamayo P., Huard C., Gaasenbeek M., Mesirov J. P., Coller H., Loh M. L., Downing J. R., Caligiuri M. A., Bloomfield C. D., Lander E. S. Molecular classification of cancer: class discovery and class prediction by gene expression monitoring. *Science* 286(5439):531-537, 1999.

Pedregosa F. et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research* 12:2825-2830, 2011.

Harris C. R. et al. Array programming with NumPy. *Nature* 585:357-362, 2020.

McKinney W. Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61, 2010.

Hunter J. D. Matplotlib: a 2D graphics environment. *Computing in Science and Engineering* 9(3):90-95, 2007.

Abadi M. et al. TensorFlow: large-scale machine learning on heterogeneous systems. arXiv:1603.04467, 2016.

Breiman L. Random forests. *Machine Learning* 45:5-32, 2001.

Kingma D. P., Ba J. Adam: a method for stochastic optimization. arXiv:1412.6980, 2014.

Srivastava N., Hinton G., Krizhevsky A., Sutskever I., Salakhutdinov R. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research* 15:1929-1958, 2014.

Ambroise C., McLachlan G. J. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences* 99(10):6562-6566, 2002.

Varma S., Simon R. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics* 7:91, 2006.

Clopper C. J., Pearson E. S. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika* 26(4):404-413, 1934.

À RETENIR

Les références de publications ont été saisies à partir de sources de référence connues de l'auteur du parcours. Le DOI et la licence du jeu Wisconsin ont été vérifiés sur la page UCI correspondante. Avant toute citation formelle, vérifiez volumes, pages et DOI dans la base bibliographique de votre établissement.